Requirement Analysis: KV-Cache Scaling

Predicting the Context Limit Before You Hit It

Functional requirements

1. Read the real architecture values (layers, KV heads, head size, vocab) from the model's config file.
2. Derive the memory model on paper: M(L, B) = weights + KV-cache + activations + overhead, with KV-cache = 2 x B x L x n_layers x n_kv_heads x d_head x bytes. Use n_kv_heads, not n_heads (GQA).
3. Commit a timestamped prediction before any measurement: maximum sequence length at batch size 1, plus memory at L = 512, 1024, 2048, 4096 and 8192.
4. Build a sweep harness that records allocated, reserved and free memory (torch.cuda and pynvml), saves to disk after every measurement, finds the limit by bisection, and stops before an out-of-memory error. Exactly one deliberate OOM is allowed.
5. Separate the components: weights (measured after load), KV-cache (slope against L) and the residual (reported as a fraction of total, with an explanation).
6. Measure the allocator gap (reserved minus allocated) and compare the maximum context with and without expandable_segments:True.
7. Compare the measured OOM threshold with the committed prediction and attribute the gap to activations, fragmentation and framework overhead. Anything unexplained is stated as unaccounted.
8. Predict the maximum context for two models and two GPUs you did not run, and verify at least one against a published figure.

Non-functional requirements

- Reproducible: pinned versions, GPU model recorded in every result.
- Crash-safe: a lost session costs at most one data point.
- Honest: the prediction is never edited after commit.

Constraints

- Model of 1B-3B parameters in fp16 (Qwen2.5-1.5B or Llama-3.2-3B), not a 7B/8B model.
- Free-tier GPU (usually a 15 GB T4), which varies between sessions.
- One deliberate OOM only.

Deliverables

- Derivation with symbols defined
- Pre-registered prediction with its commit timestamp
- Checkpointing sweep harness
- Memory-vs-length plots with the analytical curve overlaid
- Discrepancy table
- Cross-model and cross-GPU predictions with verification